

Data Structures & Algorithms Practice Questions

Formatted question bank with section headers, larger text, and styled code blocks.

Milestone 1

Question 1

Given that the truth value of the Boolean expressions A and B are True and False, respectively, what is the truth value of the Boolean expression: A and B ?

- None
- 42
- False
- True

Question 2

What is the output of the following program?

```
val = 0
for i in range(4):
    for j in range(1, 3):
        val = val + 1
print(val)
```

- 16
- 0
- 4
- 8
- 12

Milestone 1

Question 3

What is the output of the following Python code?

```
s = "gagnaskipan"  
print(s[1:4])
```

- agn
- gagna
- gagn
- agna
- gag

Question 4

Which of the following Boolean expressions is logically equivalent to the expression: not (A and B) ?

- A and (not B)
- A or B
- (not A) or (not B)
- (not A) and (not B)
- (not A) and B

Question 5

What is the output of the following Python code?

```
a = [10, 20, 30]
b = a
b.append(40)
print(a, b)
```

- [10, 20, 30, 40] [10, 20, 30, 40]
- [10, 20, 30] [10, 20, 30]
- [10, 20, 30] [10, 20, 30, 40]
- [10, 20, 30, 40] [10, 20, 30]

Milestone 1

Question 6

Which one of the following data types in Python is mutable?

- tuple
- all are mutable
- int
- list
- str

Question 7

What is the time complexity of the following function?

```
def foo(n: int):  
    val = 0  
    for i in range(n):  
        val = val + i  
        for j in range(n):  
            val = val + j  
    return val
```

- $O(n)$
- $O(n \log_2 n)$
- $O(1)$
- $O(n^2)$
- $O(\log_2 n)$

Question 8

Given two algorithms A and B with the respective time complexity of $O(n)$ and $O(n^2)$. Which of the following statements about the algorithms is always true?

- As n grows, A will eventually take longer to run than B
- A always runs faster than B
- As n grows, B will eventually take longer to run than A
- B always runs faster than A
- As n grows, one cannot tell which one will take longer to run

Milestone 1

Question 9

The table shows the running time of two algorithms, A and B, for differently sized input (n):

n	A (ms)	B (ms)
10	100	100
20	210	390
40	399	1510
80	850	6500

What is the most likely time complexity of algorithms A and B, respectively?

- $O(n)$ and $O(n^2)$
- $O(n)$ and $O(2n)$
- $O(n)$ and $O(n)$
- $O(1)$ and $O(1)$
- $O(1)$ and $O(n)$

Question 10

What is the time complexity of the following function?

```
def foo(n: int):  
    val = 0  
    for i in range(n):  
        val = val + i  
    for j in range(n):  
        val = val + j  
    return val
```

- $O(\log_2 n)$
- $O(n^2)$
- $O(n)$
- $O(n \log_2 n)$
- $O(1)$

Question 11

An algorithm A has the time-complexity of $O(n)$, where n refers to its input size. If you double the size of the input, what would one expect to happen to its running time? The running time (approximately) ...

- increases by an exponential factor
- stays the same
- reduces in half
- increases by a factor of four
- increases by a factor of two

Milestone 1

Question 12

An algorithm A has the time-complexity of $O(1)$, where n refers to its input size. If you double the size of the input, what would one expect to happen to its running time? The running time (approximately) ...

- stays the same
- reduces in half
- increases by an exponential factor
- increases by a factor of two
- increases by a factor of four

Question 13

Which one of the following statements about the algorithm's best-, average-, and worst-case time complexity is true?

```
def sort(data: list[int]):  
    n = len(data)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if data[j] < data[min_idx]:  
                min_idx = j  
        swap(data, i, min_idx) # A constant time operation.
```

- best-case > average-case = worst-case
- best-case < average-case = worst-case
- best-case < average-case < worst-case
- best-case = average-case = worst-case
- best-case = average-case < worst-case

Question 14

Which of the following statements about a dynamic array is not true?

- It can grow in size as new elements get added to it
- Its elements can be accessed directly by their index in the array
- It stores its elements continuously in memory
- It takes longer to remove an element at its front than its end
- It doubles in size every time a new element is added to it

Milestone 1

Question 15

What is the time complexity of deleting an element at the end of a dynamic array?

- $O(\log_2 n)$
- $O(n^3)$
- $O(n^2)$
- $O(n)$
- $O(1)$

Question 16

How do dynamic arrays increase their size?

- By increasing the capacity of the array already holding its elements
- By shifting all the elements in the array one step to the right
- None of the listed options is true
- By allocating a new larger array and copying existing elements over
- By allocating a second array that stores only subsequently added elements

Question 17

What does the following method (of an abstract-data-type implementing a dynamic array) return?

```
class DynArray:
    ...
    def some_method(e: object):
        for i in range(self._length):
            if e == self._data[i]:
                return i
        return ValueError('oops')
```

- the indices of all occurrences of element e in the array
- it always throws an exception
- the index of the first occurrence of element e in the array
- the element at location e in the array
- the index of the last occurrence of element e in the array

Milestone 1

Question 18

In a dynamic array with n elements (starting indexing at 0), when inserting a new element at location (index) i , one must move the already existing elements at locations _____ one step to the right (towards the end of the array).

- $i+1$ to $n-1$
- i to $n-1$
- $i+1$ to $n+1$
- $i+1$ to n
- i to n

Question 19

What is the time complexity of accessing an element at a given index in a dynamic array?

- $O(\log_2 n)$
- $O(n \log_2 n)$
- $O(1)$
- $O(n^2)$
- $O(n)$

Question 20

One can implement a dynamic array abstract data type using a fixed-sized array, where one reallocates an increasingly larger fixed-sized array when needed.

- True
- False

Stacks, Queues, Deques

Question 1

Which of the following statements is true about the following ADTs?

- Stack can be used as a Queue
- Deque can be used either as a Stack or a Queue
- Queue can be used as a Stack
- Deque can be used as a Stack, but not as a Queue
- Deque can be used as a Queue, but not as a Stack

Question 2

For abstract data types we are primarily concerned with which of the following?

- The time-complexity of the provided operations
- Which programming language they are best implemented in
- The operations provided to manipulate the type
- How their memory layout is
- All their implementation details

Question 3

Which ADT ensures the given order for insert/delete operations?

- Queue — First-In-First-Out (FIFO)
- Stack — Last-In-First-Out (LIFO)

Stacks, Queues, Deques

Question 4

The following operations are done on an empty deque, d. How will it look afterwards?

```
d.append(10)
d.append(20)
d.appendleft(30)
d.pop()
d.append(40)
d.popleft()
```

- [30, 40]
- [10, 20, 30]
- [10, 20, 30, 40]
- [30, 10, 20]
- [10, 40]

Single Linked List

Question 1

Comparing arrays and linked lists, which statement is true?

- Data in neither linked lists nor arrays is stored in contiguous memory locations
- Data in arrays is stored in contiguous memory locations but not linked lists
- Data in both linked lists and arrays is stored in contiguous memory locations
- Data in linked lists is stored in contiguous memory locations but not arrays

Question 2

The time complexity for fetching a value from a singly-linked list at a specified node (item number n in the list) is:

- $O(1)$
- $O(n)$
- $O(\log_2 n)$
- $O(n^2)$

Question 3

The time complexity for adding a new node at the front (head) of a singly-linked list is:

- $O(1)$
- $O(\log_2 n)$
- $O(n^2)$
- $O(n)$

Single Linked List

Question 4

In order to access the next-to-last node (in front of tail) in a singly-linked list the most efficient possible way for a program is to:

- Walk the entire list from the head (first node), one step at a time ($O(n)$)
- Go from the tail (last node) to its previous node in one step ($O(1)$)
- Jump from the head (first node), in steps of $n/2$ nodes at a time, where n is the number of nodes between the current location and the tail ($O(\log n)$)
- Index directly to it using its index in the list ($O(1)$)

Double Linked List

Question 1

Given a correct and efficient implementation of a list class with instance variables referencing both head (or header) and tail (or trailer), removing the last item (furthest away from head):

- is not possible in a singly-linked list, only in a doubly-linked list
- is $O(n)$ in a singly-linked list but $O(1)$ in a doubly-linked list
- is $O(n)$ in both a singly-linked list and a doubly-linked list
- is $O(1)$ in both a singly-linked list and a doubly-linked list

Question 2

Given a correct and efficient implementation of a list class with instance variables referencing both head (or header) and tail (or trailer), removing the first item (furthest away from tail):

- is $O(n)$ in a singly-linked list but $O(1)$ in a doubly-linked list
- is $O(n)$ in both a singly-linked list and a doubly-linked list
- is not possible in a singly-linked list, only in a doubly-linked list
- is $O(1)$ in both a singly-linked list and a doubly-linked list

Question 3

Given a correct and efficient implementation of a list class with instance variables referencing both head (or header) and tail (or trailer), removing an item at a specified integer (index) location in the list:

- is $O(n)$ in a singly-linked list but $O(1)$ in a doubly-linked list
- is $O(1)$ in both a singly-linked list and a doubly-linked list
- is not possible in a singly-linked list, only in a doubly-linked list
- is $O(n)$ in both a singly-linked list and a doubly-linked list

Double Linked List

Question 4

Given only a single variable referencing a particular node in a list, adding a new node into the list directly in front of the referenced node (closer to head/header):

- is $O(n)$ in a singly-linked list but $O(1)$ in a doubly-linked list
- is not possible in a singly-linked list, only in a doubly-linked list
- is $O(1)$ in both a singly-linked list and a doubly-linked list
- is $O(n)$ in both a singly-linked list and a doubly-linked list

Question 5

Given only a single variable referencing a particular node in a list, adding a new node into the list directly behind the referenced node (closer to tail/trailer):

- is not possible in a singly-linked list, only in a doubly-linked list
- is $O(1)$ in both a singly-linked list and a doubly-linked list
- is $O(n)$ in both a singly-linked list and a doubly-linked list
- is $O(n)$ in a singly-linked list but $O(1)$ in a doubly-linked list

Preparation for Exam 2

Question 1

What is the output of the following program?

```
val = 0
for i in range(4):
    for j in range(1, 3):
        val = val + 1
print(val)
```

- 0
- 4
- 8
- 12
- 16

Question 2

What is the output of the following Python code?

```
a = [10, 20, 30]
b = a
b.append(40)
print(a, b)
```

- [10, 20, 30] [10, 20, 30]
- [10, 20, 30, 40] [10, 20, 30]
- [10, 20, 30] [10, 20, 30, 40]
- [10, 20, 30, 40] [10, 20, 30, 40]

Preparation for Exam 2

Question 3

What is the time complexity of the following function?

```
def foo(n: int):  
    cnt = 0  
    for i in range(n):  
        for j in range(n):  
            cnt = cnt + 1  
    return cnt
```

- $O(1)$
- $O(\log_2 n)$
- $O(n)$
- $O(n \log_2 n)$
- $O(n^2)$

Question 4

An algorithm A has the time-complexity of $O(n)$, where n refers to its input size. If you double the size of the input, what would one expect to happen to its running time?

- reduces in half
- stays the same
- increases by a factor of two
- increases by a factor of four
- increases by an exponential factor

Question 5

For abstract data types we are primarily concerned with which of the following?

- All their implementation details
- The operations provided to manipulate the type
- The time-complexity of the provided operations
- Which programming language they are best implemented in
- How their memory layout is

Preparation for Exam 2

Question 6

Which of the following operations on a singly-linked list (implemented using both head and tail references) has the worst time-complexity?

- pop_front
- pop_back
- push_front
- push_back
- All the operations have the same time complexity

Question 7

Which ADT ensures the given order for insert/delete operations?

- Queue — First-In-First-Out (FIFO)
- Stack — Last-In-First-Out (LIFO)

Question 8

Which of the following statements is true about the following ADTs?

- Stack can be used as a Queue
- Deque can be used either as a Stack or a Queue
- Queue can be used as a Stack
- Deque can be used as a Stack, but not as a Queue
- Deque can be used as a Queue, but not as a Stack

Preparation for Exam 2

Question 9

The following operations are performed on an empty stack, named `s`. How does the stack (its top is to the right) look afterwards?

```
s.push(10)
s.push(20)
s.push(30)
s.pop()
s.push(s.top())
s.push(40)
```

- [10, 20, 30, 40]
- [10, 20, 20, 40]
- [10, 20, 40]
- [40, 30, 20, 10]
- []

Question 10

The following operations are performed on a queue, named `q`, already holding the elements `[10, 20]`. How does the queue (its front is to the left, back to the right) look afterwards?

```
q.enqueue(30)
q.dequeue()
q.dequeue()
q.enqueue(40)
```

- `[]`
- `[30]`
- `[10, 40]`
- `[30, 40]`
- `[40, 30]`

Preparation for Exam 2

Question 11

The following operations are performed on an empty deque, named d. How does the deque (its front is to the left, back to the right) look afterwards?

```
d.append(10)
d.appendleft(30)
d.appendleft(40)
d.pop()
d.appendleft(d.back())
```

- [30, 40, 30]
- [40, 30, 40]
- [30, 40]
- [40, 30]
- []

Question 12

Given a single-linked list, implemented using head and tail references, which code fragment correctly adds a new element item to the beginning of a (possibly empty) list?

- self.head = Node(item, None)
- self.head = Node(item, self.head)
- node = Node(item, self.head); self.head = node; self.tail = node
- node = Node(item, self.head); self.head = node; if not self.tail is None: self.tail = None
- node = Node(item, self.head); self.head = node; if self.tail is None: self.tail = node

Question 13

Given a double-linked list, implemented using nodes with prev and next references, which code fragment correctly removes the node at location curr from the list?

- `curr.prev = curr.next ; curr.next = curr.prev`
- `curr.prev.prev = curr.prev ; curr.next.next = curr.next`
- `curr.prev.next = curr.next ; curr.next.prev = curr.prev`
- `curr.prev.prev = curr.next ; curr.next.next = curr.prev`
- `curr = None`

Preparation for Exam 2

Question 14

The following code is supposed to print the elements of a single-linked list. Unfortunately, it does not work as intended. What is the problem?

```
def print_list(self):  
    node = self.head  
    while node is not self.tail:  
        print(node.item)  
        node = node.next
```

- crashes given an empty list
- prints all but the last item in the list
- prints all but the first item in the list
- prints every other item in the list
- loops forever

Recursion

Question 1

When designing a recursive function, one must ensure that _____.

- the base case is never executed
- no more than one recursive call chain ends in a base case
- at least one recursive call chain ends in a base case
- all recursive call chains end in a base case

Question 2

Which of the following statements about recursive functions is not true?

- It must have exactly one call to itself.
- It can have more than one calls to itself.
- If correct, it must eventually return.
- If correct, it must include one or more base cases.

Question 3

The following recursive function is given. What happens if it is called with a negative number as an argument, e.g. factorial(-1)?

```
def factorial(n: int):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

- It returns -1
- It returns 1
- It runs forever
- It eventually terminates with an error (exception)

Recursion

Question 4

Which of the following statements about recursive vs. iterative solutions is not true?

- Recursion can result in more readable solutions.
- It is almost always preferable to use recursion, if possible.
- Iterative solutions are preferable if there is a risk of very long recursive call chains.
- Iterative solutions generally have less overhead than recursive ones, and might thus run slightly faster.

Question 5

What does the following code output?

```
def foo(d: int) -> int:
    if d <= 0:
        return 1
    cnt = 0
    cnt += foo(d-1)
    cnt += foo(d-1)
    return cnt

print(foo(2))
```

- 0
- 1
- 4
- 8

Recursion II

Question 1

Look at this recursive program. What does it do?

```
def foo(a_str, n):  
    if n == 0:  
        return 0  
    n -= 1  
    if a_str[n] == 'a':  
        return 1 + foo(a_str, n)  
    return foo(a_str, n)
```

- Returns the length of the string a_str if 'a' occurs in it, otherwise 0
- Recurs endlessly and never returns
- Returns 1 if 'a' occurs in the first n letters of a_str, otherwise 0
- Returns the number of times 'a' occurs in the first n letters of a_str

Question 2

A programmer is asked to make a function that returns the sum of the first n natural numbers. How will this code behave for print(bar(6))?

```
def bar(n):  
    if n == 1:  
        return n  
    return n + bar(n)
```

- It will return the sum of the first n-1 natural numbers
- It will recur endlessly and never return
- It will return 1
- It will return the sum of the first n natural numbers

Recursion II

Question 3

Same code as previous question. What change is needed for it to work?

```
def bar(n):  
    if n == 1:  
        return n  
    return n + bar(n)
```

- In the recursive call we need to call bar(n-1) instead of bar(n)
- No change, it works correctly already
- In the if condition, we should check if n == 0 instead of n == 1
- If the condition, n == 1, is true it should return 0 instead of n

Question 4

A programmer is asked to make a function that calculates n to the power of m. How will this code behave for print(bar(2, 3))?

```
def bar(n, m):  
    return n * bar(n, m-1)
```

- It will return 2
- It will return 6
- It will return 8
- It will recur endlessly and never return

Recursion II

Question 5

Same code as previous question. What change is needed for it to work?

```
def bar(n, m):  
    return n * bar(n, m-1)
```

- In the recursive call we need to call bar(n-1, m) instead of bar(n, m-1)
- In the recursive call we need to call bar(n, m+1) instead of bar(n, m-1)
- We need to add a condition: if m == 0: return 1
- No change, it works correctly already

Tree

Question 1

Given the tree in the referenced image, which traversal will give the order A, B, C, D, E, F, G?

- Inorder
- None of the other options will give this order
- Postorder
- Preorder

Question 2

Given the tree in the referenced image, which traversal will give the order C, D, B, A, F, G, E?

- None of the other options will give this order
- Preorder
- Postorder
- Inorder

Question 3

In a binary tree of height 4, what is the maximum height of the left subtree of a node?

- 2
- 4
- 3
- 1

Tree

Question 4

What is the maximum number of nodes at a binary tree's n -th level (nodes of depth n , root is considered level 0)?

- 2^n
- 2^{n-1}
- 2^{n+1}
- 3^n

Binary Search Tree

Question 1

Look at the image of a binary search tree. What is the output of printing its contents inorder?

- 2 3 5 7 8 9
- 7 3 2 5 9 8
- 7 3 9 2 5 8
- 3 2 5 7 9 8

Question 2

If the item '4' were added to the BST in the image, where would it end up in the tree?

- Left child on '8'
- Left child on '5'
- Right child on '2'
- Right child on '9'

Question 3

In a correct and full binary search tree what is the worst-case time complexity of finding a particular item (where n is the number of items in the entire tree)?

- $O(n^2)$
- $O(n)$
- $O(1)$
- $O(\log n)$

Binary Search Tree

Question 4

If data is entered into a BST in a sequence ordered by key values, what will the worst-case time complexity of finding an item in the resulting tree be (where n is the number of items in the entire tree)?

- $O(\log n)$
- $O(1)$
- $O(n^2)$
- $O(n)$

Question 5

What is the worst-case time complexity of outputting every item in a BST, ordered by key values (where n is the number of items in the entire tree)?

- $O(n^2)$
- $O(n)$
- $O(\log n)$
- $O(n * \log n)$

Maps and Hash Functions

Question 1

Which of the following is another common name for a map?

- Stack
- Queue
- Dictionary
- Heap

Question 2

When implementing a map, which data structure is best suited for achieving the best average-case time complexity for its core operations?

- Sorted array
- Linked list
- Binary search tree
- Hash table

Question 3

What is the purpose of a hash function?

- Comparing two keys for equality
- Converting a key into an integer value in a fixed range
- Making a hash brownie
- Resizing an array of keys

Maps and Hash Functions

Question 4

A typical hash function is composed of two parts. Which are they?

- Sorting and searching
- Generation and compression of a hash code
- Encoding and decoding a hash code
- Resizing and copying

Question 5

The load factor of a hash table, λ , is defined as? (n is number of items in the hash table and N is the size of the hash-table array)

- n / N
- N / n
- $n * N$
- $\log n / N$

Priority Queues

Question 1

What is the worst-case time complexity of removing a maximum element from a binary max-heap with n elements?

- $O(1)$
- $O(\log n)$
- $O(n)$
- $O(n \log n)$

Question 2

What is the worst-case time complexity of inserting an element into a binary heap with n elements?

- $O(1)$
- $O(\log n)$
- $O(n)$
- $O(n \log n)$

Question 3

After calling `remove_max` on a max-heap, which element is typically moved to the root before restoring the heap property?

- The smaller of the two children of the root
- The larger of the two children of the root
- The rightmost element of the deepest level
- The element with the next-largest key

Priority Queues

Question 4

Which of the following best describes a complete binary tree?

- Every node has either 0 or 2 children
- Every level is fully filled
- Every level is fully filled, except possibly the deepest, which is filled left to right
- All leaves are at the same depth

Question 5

Which binary tree type is used to represent a binary heap stored in an array?

- A complete binary tree
- A perfect binary tree
- A full binary tree
- A proper binary tree